SmartRetail Logic Challenge

</> RETAIL DOMAIN HACKATHON

JAVA · PYTHON

A progressive, industry-grade coding challenge built around the fictional retail chain **TrendMart** — covering Conditional Statements, Nested Ifs, and Loops at Intermediate to Advanced levels.

*Developed by **Talenciaglobal***

Hackathon at a Glance

5

Challenge Levels

From basic conditionals to full system integration

4-6

Hours Duration

Self-paced or team-based format

2

Languages

Java or Python — participant's choice

40%

Logic Weight

Correctness & logic is the top judging criterion

According to industry research, **over 73% of retail enterprises** now rely on automated pricing and inventory logic systems — making these skills among the most commercially valuable in software development today. This hackathon bridges academic programming with real-world retail engineering.

Hackathon Objective

Participants will build **intelligent retail logic modules** for the fictional chain **TrendMart**. Each level progressively adds complexity to core retail operations including pricing, inventory management, customer offers, sales processing, and analytics.

Clean Code Standards

All submissions must be well-commented, efficiently structured, and follow professional coding conventions.

Proper Input Handling

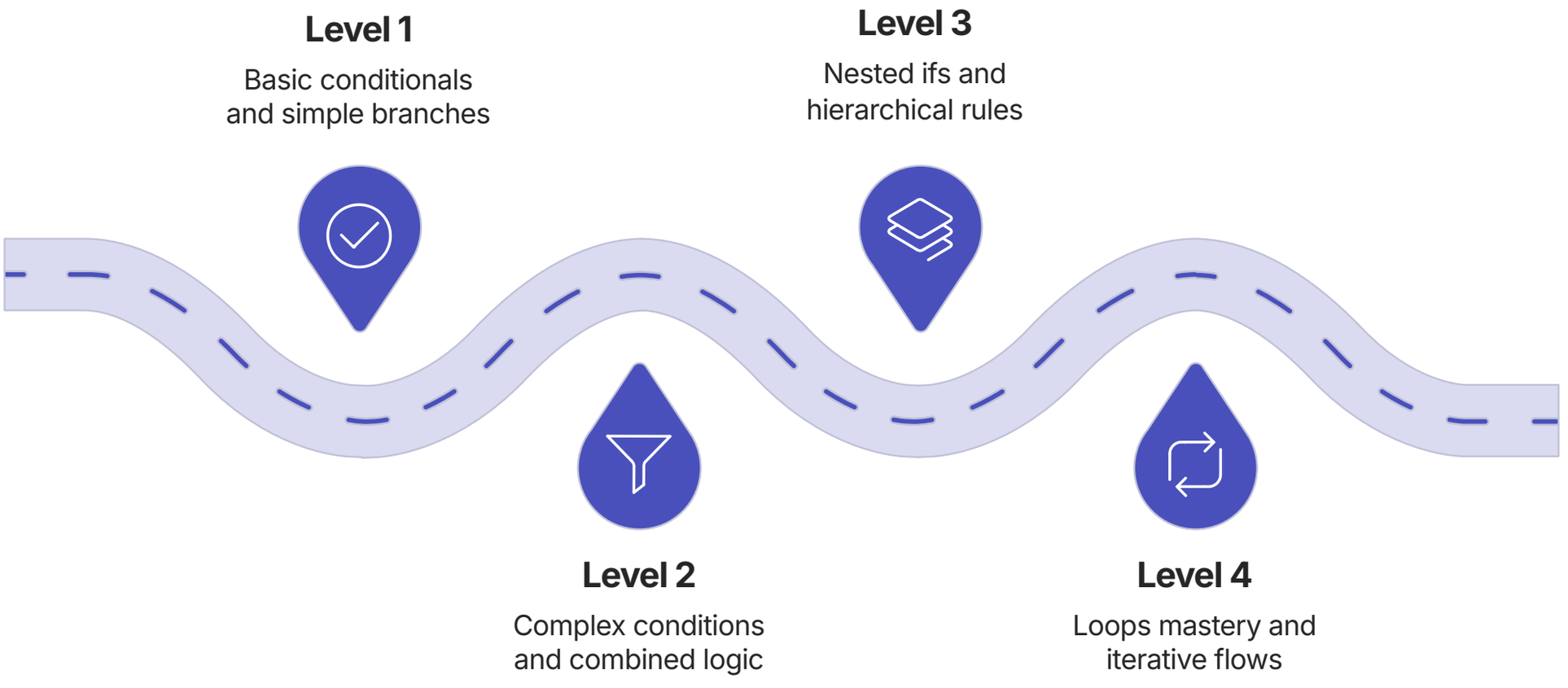
Robust validation and edge-case management are expected at every level of the challenge.

Formatted Output

Results must be presented with proper rounding, labeling, and human-readable formatting.

Challenge Progression Overview

The five levels form a deliberate learning arc — from foundational conditional logic to a fully integrated retail processing engine. Each level builds directly upon the last.



This architecture mirrors how enterprise retail systems are actually built — starting with atomic pricing rules and culminating in end-to-end transaction processors. **Retail technology is a $25B+ global market**, and logic-layer engineering is at its core.

Level 1: Smart Pricing Engine

BEGINNER–INTERMEDIATE | BASIC CONDITIONALS

Implement a **dynamic pricing calculator** that applies category-based taxes, customer-type discounts, and bulk purchase rules — while enforcing a minimum price floor.

Tax Rules by Category

- Electronics: +18% tax
- Apparel: +12% tax
- Grocery: +5% tax
- Beauty: +15% tax

Discount & Floor Rules

- Premium customer: 8% discount
- VIP customer: 15% discount
- Bulk (qty > 10): additional 5% off
- Price floor: never below 60% of base price

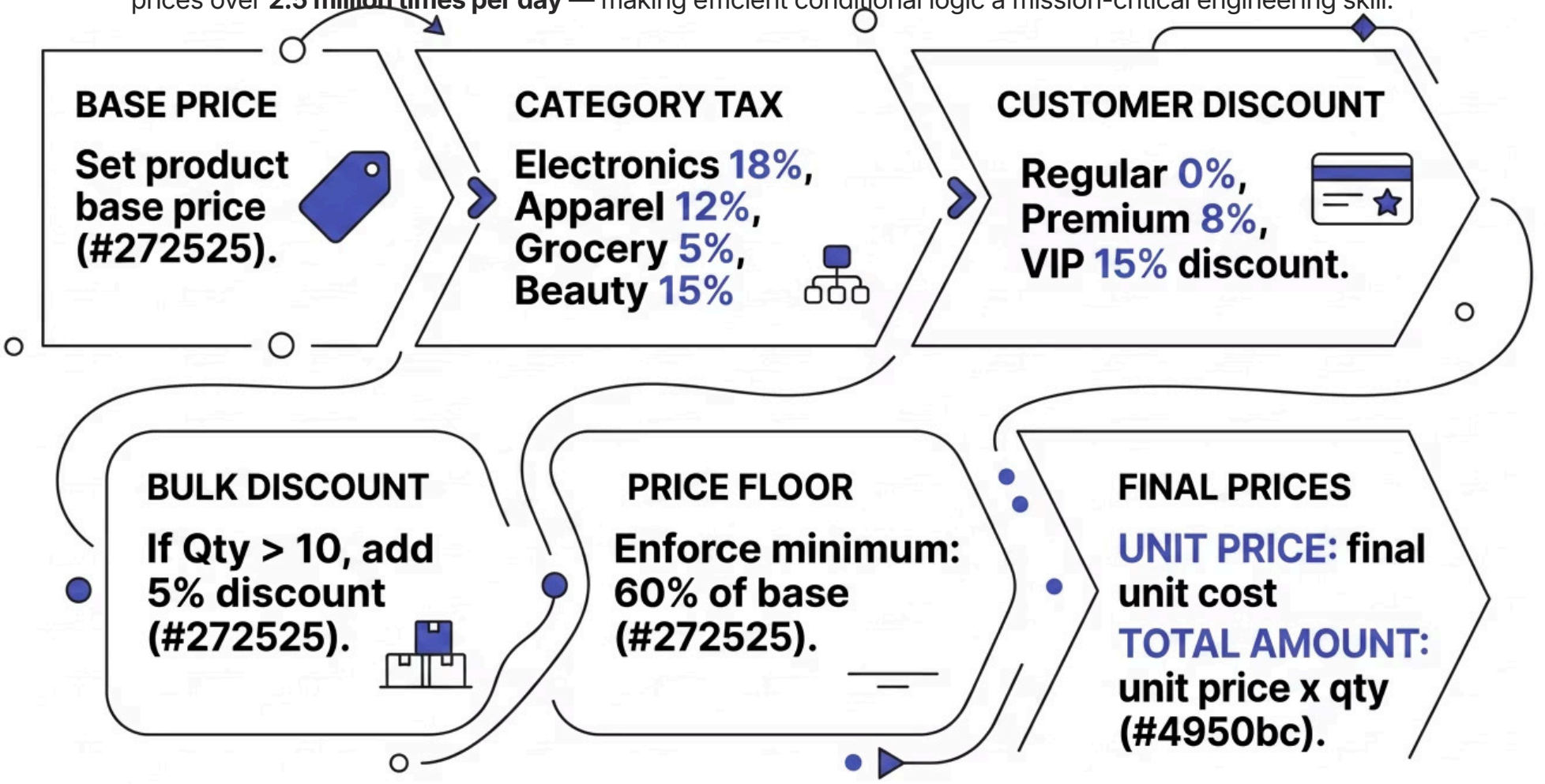
Inputs Required

product_category, base_price, quantity, customer_type (Regular / Premium / VIP)

Thinking Trigger: How do you prioritize which discount or tax applies first? Consider the order of operations carefully — applying tax before or after discount yields different results.

Level 1: Pricing Logic Flow

Industry context: Dynamic pricing engines process **millions of SKU-level calculations daily** in large retail chains. Amazon alone adjusts prices over **2.5 million times per day** — making efficient conditional logic a mission-critical engineering skill.



Level 2: Loyalty & Offer Engine

INTERMEDIATE | COMPLEX CONDITIONALS + LOGICAL OPERATORS

Create a **personalized offer generator** for checkout using complex if-elif-else chains with `and`, `or`, and `not` operators. The engine must evaluate multiple overlapping conditions and resolve offer conflicts intelligently.

Input Parameters

total_cart_value · membership_years · items_count · has_coupon · is_weekend · purchase_history (High / Medium / Low)

1

Premium Bundle Offer

cart_value > 5000 **AND** membership_years ≥ 2

2

Free Shipping + 10% Off

items_count > 8 **OR** (cart_value > 3000 **AND** is_weekend)

3

Extra 12% Coupon Discount

has_coupon **AND** purchase_history == "High" — cannot combine with free shipping

4

VIP Elite Reward

cart_total > 10000 **AND** membership_years > 5 → flat 20% + gift voucher

⚠ **Thinking Trigger:** How do you handle conflicting offers and priority? Define a clear precedence hierarchy before writing a single line of code.

Level 2: Offer Priority Matrix

Understanding offer conflict resolution is essential. The table below summarizes how competing offers interact and which takes precedence in each scenario.

Condition Met	Premium Bundle	Free Ship + 10%	Coupon 12%	VIP Elite 20%
cart > 10000, years > 5	Overridden	Overridden	Overridden	✅ Applied
cart > 5000, years ≥ 2	✅ Applied	Stackable	Stackable	—
has_coupon + High history	Stackable	❌ Conflict	✅ Applied	—
items > 8 or weekend cart	Stackable	✅ Applied	❌ Conflict	—

Retail loyalty programs that implement intelligent offer stacking report **up to 34% higher average order values** compared to single-offer systems, according to McKinsey retail analytics benchmarks.

Level 3: Checkout Validator & Fraud Detection

INTERMEDIATE–ADVANCED NESTED IFS

Build a **Checkout Validator with Nested Risk Assessment** — the most architecturally complex level of the challenge. The system must evaluate payment method eligibility, fraud risk windows, and produce a final approval decision.

Inputs Required

cart_total · payment_method (Card, UPI, Cash, EMI) · customer_risk_score (0–100) · items_list (list of categories) · time_of_purchase (HH:MM)

EMI Path

- cart_total < 8000 → Not Eligible
- risk_score > 40 → High Risk, Approval Needed
- Else → Eligible + interest calculation

Card Path

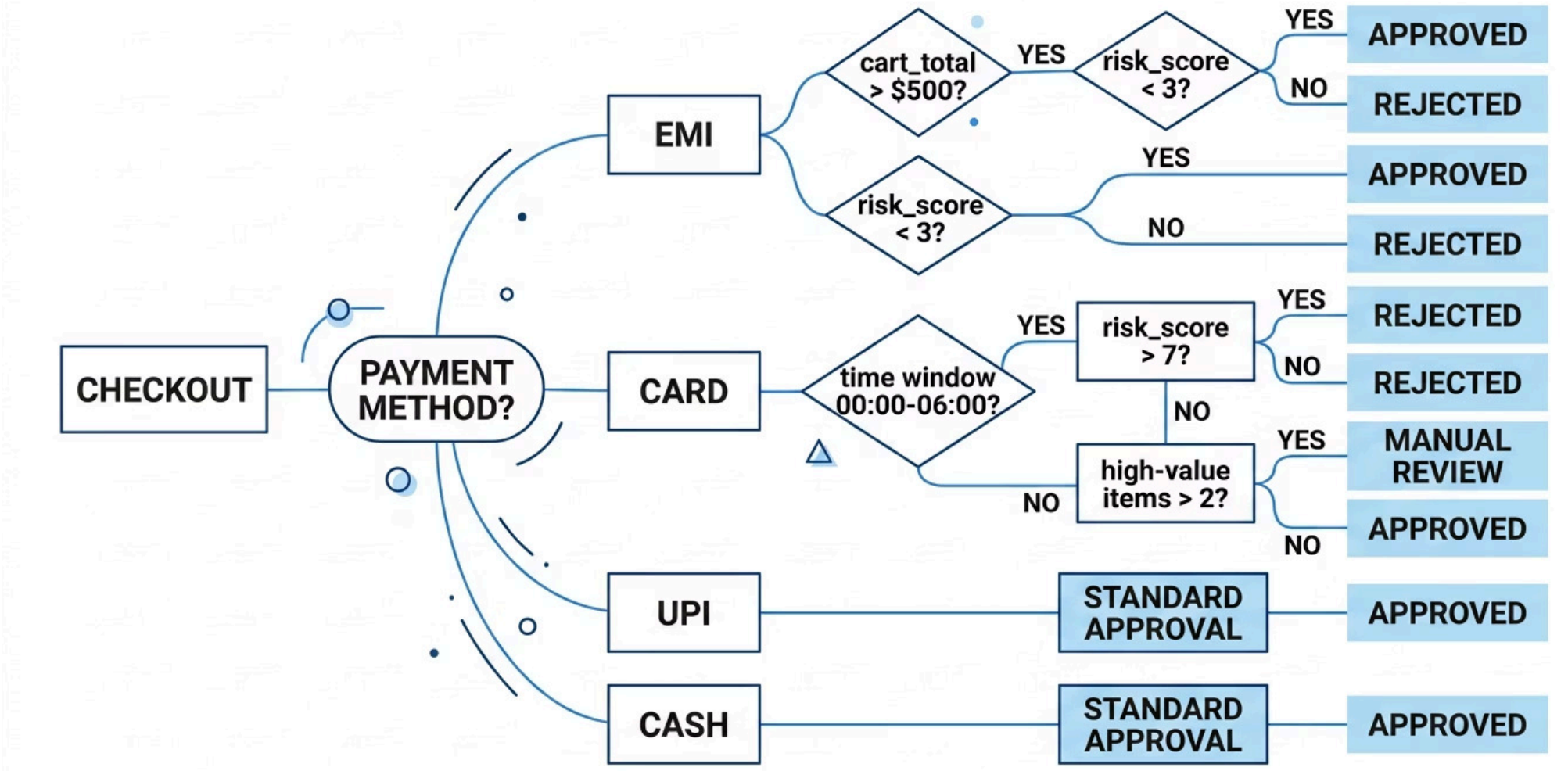
- Fraud window: 00:00–06:00
- Cross-check risk_score
- Flag high-value items (>₹2000 each, count > 2)

Final Decision

- ✅ Approved
- ❌ Rejected
- ⚠️ Manual Review

Combined with Level 2 discount logic

Level 3: Nested Logic Architecture



Thinking Trigger: How deep should your nested structure go without becoming unreadable? Consider early returns or guard clauses to flatten deeply nested logic. Industry best practice recommends a maximum nesting depth of 3 levels before refactoring.

Global retail fraud losses exceeded **$100 billion in 2023** (NRF Retail Security Survey). Real-time transaction risk scoring — exactly what this level simulates — is the primary defense mechanism deployed by major payment processors worldwide.

Level 4: Inventory & Reorder Optimizer

ADVANCED LOOPS MASTERY

Build a **Smart Inventory Management System** using creative application of `for` and `while` loops. This level tests the participant's ability to process collections efficiently and derive actionable business intelligence.

Product Input Schema

product_name · current_stock · min_threshold · daily_sales_avg · lead_time_days · category

Stock Classification

Loop through all products and classify as **Critical** (below threshold), **Low** (threshold to 2× threshold), or **Healthy**.

Reorder Quantity

For Critical and Low items: `reorder_qty = (daily_sales_avg × lead_time_days × 1.5) − current_stock`

Category Analysis

Count products needing reorder per category and identify the category with highest reorder urgency.

7-Day Sales Projection

Simulate daily sales deduction using nested loops and output projected stockout dates for at-risk products.

Level 4: Loop Strategy & Efficiency

Single-Pass Approach

Classify stock status, compute reorder quantities, and accumulate category counters — all within one loop iteration per product.

- O(n) time complexity
- Minimal memory overhead
- Preferred for large catalogs
- Requires careful variable initialization

Multi-Pass Approach

Separate loops for classification, reorder calculation, and analytics. Easier to read but less efficient at scale.

- O(n × k) time complexity
- Higher readability for beginners
- Acceptable for small datasets
- Refactor to single-pass for production

Thinking Trigger: Can you accomplish all four tasks in a single pass? This is the efficiency challenge at the heart of Level 4.

Retail giants like Walmart process inventory data for over **500 million SKUs globally**. Loop optimization at scale is not academic — a single-pass algorithm on a 1M-product catalog can reduce processing time by **60–80%** compared to naive multi-pass implementations.

Level 5: TrendMart Daily Sales Processor

ADVANCED+ FULL SYSTEM INTEGRATION

The capstone challenge: build the **TrendMart Daily Sales Processor** — a complete end-to-end system that integrates all prior levels into a single cohesive retail engine.

1

Transaction Input

Accept multiple customer transactions as list of dicts or class instances

2

Per-Transaction Processing

Apply Level 1 pricing → Level 2 offers → Level 3 fraud validation

3

Inventory Update

Run Level 4 inventory loop after all transactions are processed

4

Summary Report

Generate full analytics: revenue, top category, fraud alerts, discount leaders

Level 5: Summary Report Requirements

The final summary report generated by the processor must include the following business intelligence outputs — mirroring what a real retail operations dashboard would surface at end-of-day.

Total Revenue

Aggregate net revenue across all processed transactions after discounts and taxes

Top Performing Category

Category generating the highest revenue contribution in the session

Critical Stock Products

All products that hit critical inventory threshold during the processing run

Highest Discount Given

Maximum discount applied and the corresponding customer type that received it

Fraud & Review Alerts

All transactions flagged for manual review or rejected by the fraud detection module

Level 5: Advanced Challenges

For top-performing teams seeking to differentiate their submissions, the following advanced engineering challenges are available. These mirror real-world software architecture decisions made in production retail systems.

1

Object-Oriented Design

Implement the system using OOP principles with dedicated `Product`, `Customer`, and `Transaction` classes. Encapsulate business logic within appropriate methods.

2

In-Memory Data Persistence

Simulate data persistence using in-memory structures (dictionaries, lists). Maintain state across multiple transaction batches within a single session.

3

Large Dataset Optimization

Optimize loops and conditional logic to handle **1,000+ transactions** efficiently. Profile and reduce unnecessary iterations.

4

Error Handling & Edge Cases

Implement comprehensive error handling for negative stock values, invalid inputs, missing fields, and boundary conditions across all modules.

Judging Criteria

Submissions will be evaluated across five dimensions, weighted to reflect the priorities of professional software engineering in a retail technology context.

40%

Correctness & Logic

Does the code produce accurate outputs for all test cases, including edge cases?

20%

Code Quality & Readability

Is the code clean, well-commented, and structured for maintainability?

15%

Efficiency

Are loops optimized? Is deep nesting avoided where guard clauses would suffice?

15%

Edge Case Handling

Does the system gracefully handle invalid inputs, boundary values, and unexpected states?

10%

Creativity in Retail Features

Does the solution demonstrate innovative thinking in applying retail domain logic?

Submission Format & Requirements

File Naming Convention

Submit a single main file:

- Python: `smart_retail.py`
- Java: `SmartRetail.java`

Code Documentation

Every logical block must include clear inline comments explaining the decision rationale — not just what the code does, but *why*.

Test Cases Required

Each level must include sample inputs and expected outputs demonstrating:

- Standard happy-path scenarios
- Boundary condition tests
- Invalid input handling
- Edge cases (zero stock, max discount, fraud window)

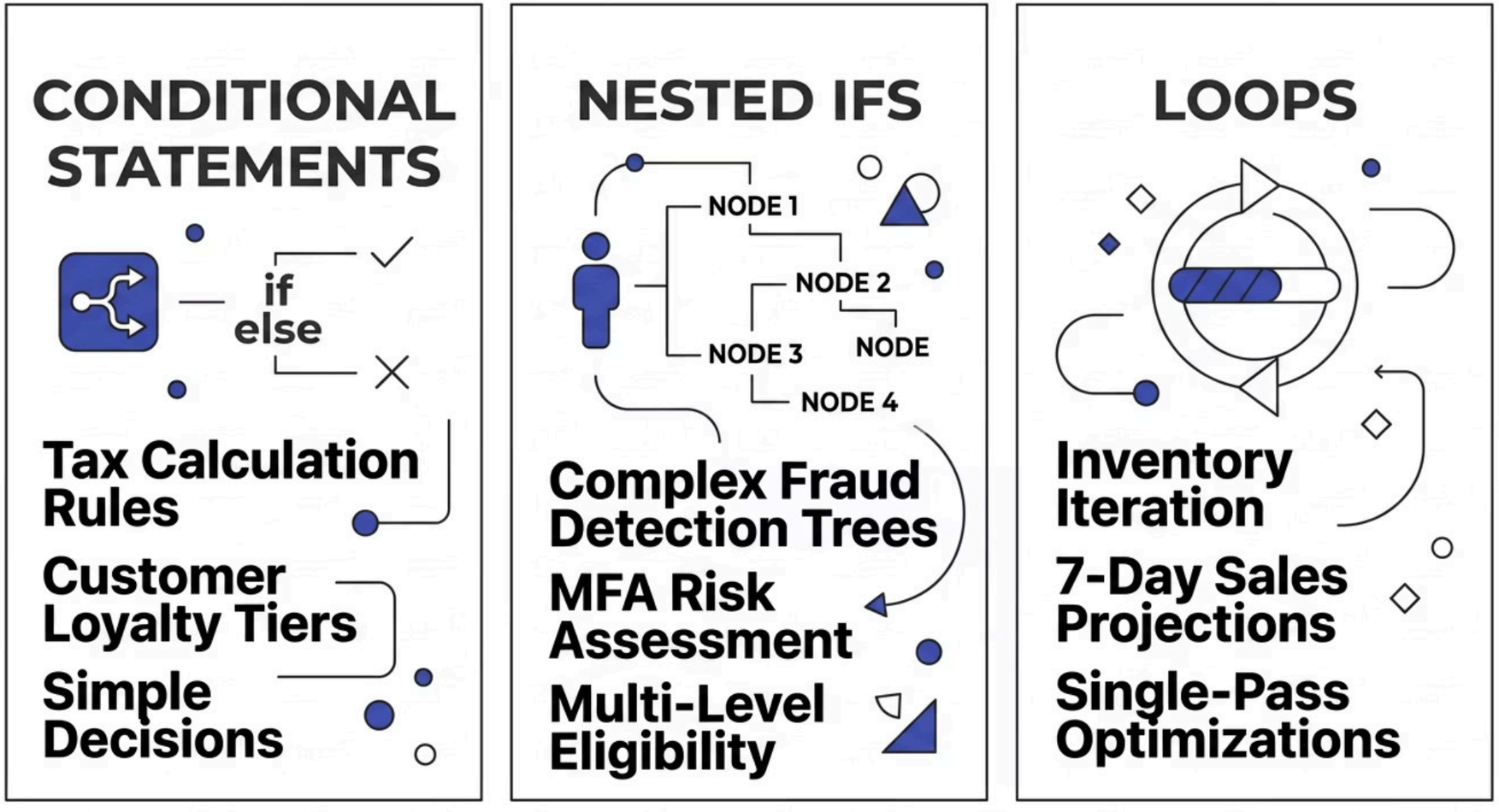
Output Formatting

All monetary values must be rounded to 2 decimal places. Reports must be clearly labeled and human-readable.

Industry standard: Professional code review processes at top retail tech firms (Shopify, Flipkart, Amazon) evaluate **readability and documentation** as heavily as functional correctness during technical assessments.

Key Programming Concepts Reinforced

This hackathon is deliberately structured to build deep, transferable competency in three foundational programming constructs — the same constructs that underpin virtually every enterprise business logic system.



A 2023 Stack Overflow Developer Survey found that **conditional logic and loop optimization** are cited by over **61% of backend engineers** as the most frequently applied skills in day-to-day production code — validating the practical focus of this challenge design.

Industry Relevance & Real-World Context

Every module in this hackathon maps directly to a real system component deployed in modern retail technology stacks.

Dynamic Pricing Engines

Used by Amazon, Flipkart, and Walmart to adjust prices in real time. Amazon's pricing engine executes **over 2.5 million price changes per day** using conditional logic at scale.

Loyalty & Personalization Systems

Starbucks' loyalty program — powered by offer-stacking logic — drives **53% of total US revenue**. Personalized offer engines are among the highest-ROI investments in retail tech.

Real-Time Fraud Detection

Visa and Mastercard process **over 65,000 transactions per second** through nested risk-scoring logic. Fraud detection trees are mission-critical infrastructure in every payment system.

Automated Inventory Management

Zara's inventory loop system enables **new product cycles every 2 weeks**. Loop-driven reorder optimization reduces overstock costs by an average of **30%** in fast-fashion retail.

Ready to Build TrendMart?

The **SmartRetail Logic Challenge** is more than a hackathon — it is a structured immersion into the logic systems that power the global retail industry. Participants who complete all five levels will have built a functional retail processing engine and demonstrated mastery of the conditional and iterative programming patterns that define professional-grade software engineering.

Start with Level 1	Progress Through Levels 2–4	Integrate at Level 5
Master the pricing engine and establish your conditional logic foundation	Build offer logic, fraud detection, and inventory optimization incrementally	Combine everything into the TrendMart Daily Sales Processor

> *"The best retail engineers don't just write code that works — they write code that scales, reads clearly, and handles the unexpected gracefully."*

Developed by Talenciaglobal · SmartRetail Logic Challenge · Java & Python · Intermediate to Advanced